

ROB521: Mobile Robotics and Perception

Assignment #3: Occupancy Grid Mapping & Particle Filter Localization

Student: Azure Lin

Question 1: Occupancy Grid Mapping

The occupancy grid is built from noise-free ground-truth poses using a log-odds inverse sensor model. For each laser beam, cells along the ray before the endpoint receive a free-space update ($\ell_{\text{free}} = -0.4$), while the endpoint cell receives an occupied update ($\ell_{\text{occ}} = 0.85$). Log-odds are clamped to $[-5, 5]$.

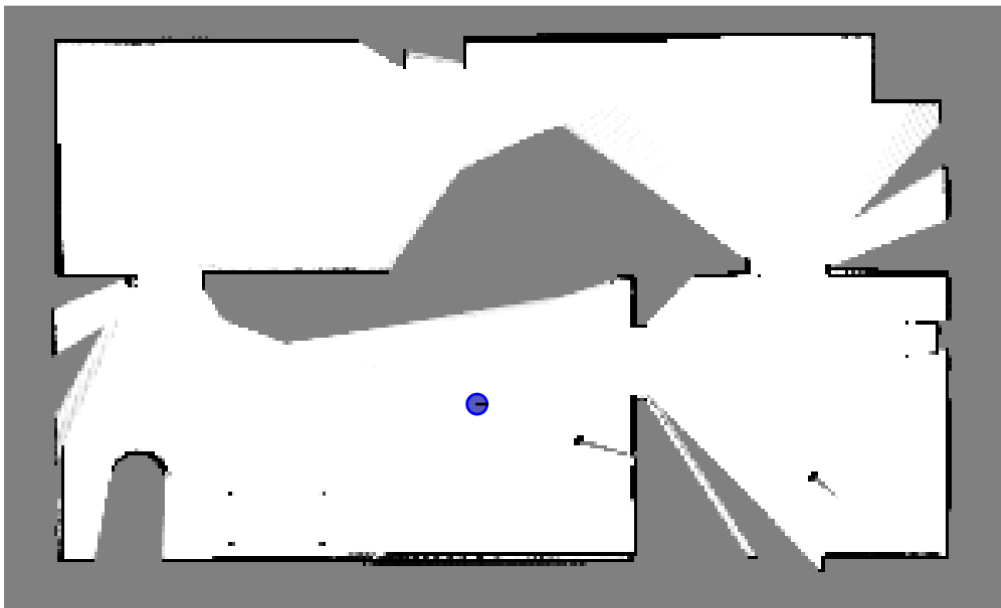


Figure 1: Occupancy grid map built from ground-truth poses and laser scans.

Two observations from Figure 1:

1. **Walls converge quickly.** Because the log-odds update is additive, repeated observations of the same wall accumulate rapidly toward the clamp limits, producing sharp black–white boundaries.
2. **Unexplored regions remain gray.** Some open areas stay at $p = 0.5$ (gray) because the robot's trajectory never sent a beam through them. The occupancy grid is passive. It only records what is observed. This motivates active exploration (e.g. frontier-based planning) to reduce map uncertainty.

Question 2: Particle Filter Localization

The particle filter corrects noisy odometry ($\sigma_v = 0.2$ m/s, $\sigma_\omega = 0.04$ rad/s) using two laser beams (extreme left and right) matched against the occupancy grid. Each iteration:

1. **Predict.** Each particle is propagated via the noisy unicycle model with additional lateral velocity noise ($\sigma_u = 0.2$ m/s) to account for unmodelled slip:

$$\begin{aligned}x_{k+1}^{(n)} &= x_k^{(n)} + \Delta t (\tilde{v} \cos \theta_k^{(n)} - \tilde{u} \sin \theta_k^{(n)}) \\y_{k+1}^{(n)} &= y_k^{(n)} + \Delta t (\tilde{v} \sin \theta_k^{(n)} + \tilde{u} \cos \theta_k^{(n)}) \\\theta_{k+1}^{(n)} &= \theta_k^{(n)} + \Delta t \tilde{\omega}\end{aligned}$$

2. **Update.** For each beam, a ray is cast from the particle pose through the occupancy grid until hitting a cell with $p > 0.5$. The weight is the Gaussian likelihood of the residual between measured and expected range:

$$w^{(n)} \propto \prod_{j \in \{L, R\}} \exp\left(-\frac{(r_j^{\text{meas}} - r_j^{\text{exp}})^2}{2\sigma_{\text{laser}}^2}\right)$$

with $\sigma_{\text{laser}} = 0.5$ m.

3. **Resample.** Systematic resampling eliminates low-weight particles while preserving diversity better than multinomial resampling.

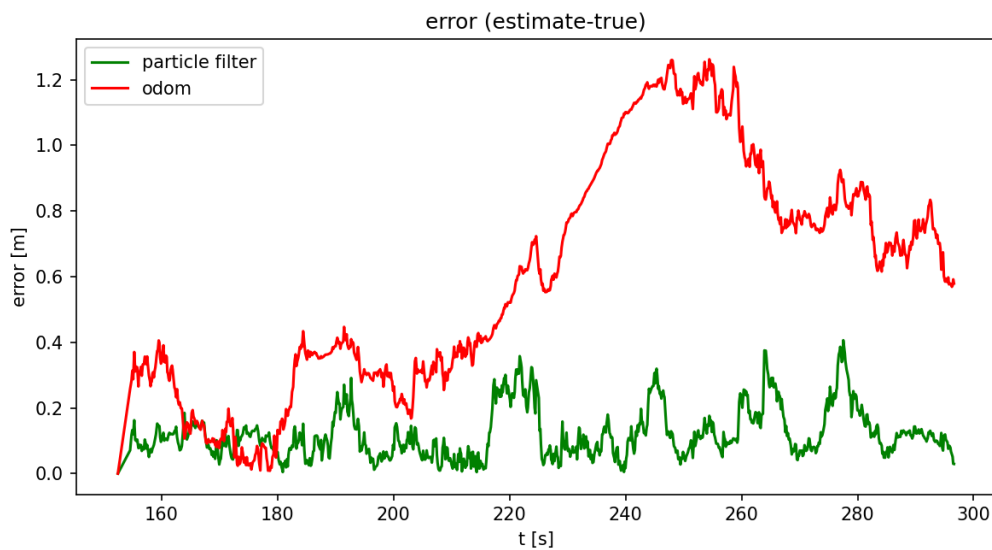


Figure 2: Position error: particle filter (green) vs. odometry only (red).

Figure 2 compares the error of the particle filter against noisy odometry alone.

The odometry error grows as heading noise accumulates, peaking at about 1.2 m mid-trajectory, then partially narrows when the robot loops back and the drift cancels by chance.

The particle filter stays below 0.3 m throughout: each measurement update corrects drift before it compounds. Two beams suffice because they constrain different directions simultaneously.

Appendix: Code

Question 1: Occupancy Grid Mapping

```
# =====
# ass3_q1.py
# =====
#
# This assignment will introduce you to the idea of first building an
# occupancy grid then using that grid to estimate a robot's motion
# using a
# particle filter.
#
# There are two questions to complete (5 marks each):
#   Question 1: code occupancy mapping algorithm
#   Question 2: see ass3_q2.py
#
# Fill in the required sections of this script with your code, run it
# to
# generate the requested plot & movie, then paste the plots into a
# short report
# that includes a few comments about what you've observed. Append your
# version of this script to the report. Hand in the report as a PDF
# file
# and the two resulting .mp4 files from Questions 1 and 2.
#
# requires: numpy, scipy, matplotlib, opencv-python, and the data file
# 'gazebo.mat'
#
# Steven Waslander March 2026

import numpy as np
from scipy.io import loadmat
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import cv2
from pathlib import Path

# Set random seed for repeatability
np.random.seed(1)

# =====
# Load the dataset from file
# =====
#   ground truth poses: t_true x_true y_true theta_true
#   odometry measurements: t_odom v_odom omega_odom
#   laser scans: t_laser y_laser
#   laser range limits: r_min_laser r_max_laser
#   laser angle limits: phi_min_laser phi_max_laser

data = loadmat('gazebo.mat')

t_true = data['t_true'].flatten()
x_true = data['x_true'].flatten()
y_true = data['y_true'].flatten()
theta_true = data['theta_true'].flatten()
t_odom = data['t_odom'].flatten()
v_odom = data['v_odom'].flatten()
```

```

omega_odom = data['omega_odom'].flatten()
t_laser = data['t_laser'].flatten()
y_laser = data['y_laser']
r_min_laser = data['r_min_laser'][0, 0]
r_max_laser = data['r_max_laser'][0, 0]
phi_min_laser = data['phi_min_laser'][0, 0]
phi_max_laser = data['phi_max_laser'][0, 0]

# =====
# Question 1: build an occupancy grid map
# =====
#
# Write an occupancy grid mapping algorithm that builds the map from
# the
# perfect ground-truth localization. Some of the setup is done for you
# below. The resulting map should look like "ass2_q1_soln.png". At the
# end you will save your occupancy grid map to the file "occmap.mat"
# for
# use in Question 2 of this assignment.

# Allocate a big 2D array for the occupancy grid
ogres = 0.05 # resolution of occ grid
ogxmin = -7 # minimum x value
ogxmax = 8 # maximum x value
ogymin = -3 # minimum y value
ogymax = 6 # maximum y value
ognx = int((ogxmax - ogxmin) / ogres) # number of cells in x
direction
ogny = int((ogymax - ogymmin) / ogres) # number of cells in y
direction
oglo = np.zeros((ogny, ognx)) # occupancy grid in log-odds format
ogp = 0.5 * np.ones((ogny, ognx)) # occupancy grid in probability
format

# Precalculate some quantities
numodom = t_odom.shape[0]
npoints = y_laser.shape[1]
angles = np.linspace(phi_min_laser, phi_max_laser, npoints)
dx = ogres * np.cos(angles)
dy = ogres * np.sin(angles)

# Interpolate the noise-free ground-truth at the laser timestamps
t_interp = np.linspace(t_true[0], t_true[numodom - 1], numodom)
x_interp = interp1d(t_interp, x_true, kind='linear')(t_laser)
y_interp = interp1d(t_interp, y_true, kind='linear')(t_laser)
theta_interp = interp1d(t_interp, theta_true, kind='linear')(t_laser)
omega_interp = interp1d(t_interp, omega_odom, kind='linear')(t_laser)

# Set up the video writer
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
video_size = (4 * ognx, 4 * ogny)
vid = cv2.VideoWriter('ass2_q1.mp4', fourcc, 10.0, video_size)

# Initialize figure for plotting
fig = plt.figure(1, figsize=(10, 8))
plt.clf()
plt.imshow(1 - ogp, cmap='gray', origin='lower')
plt.axis('equal')
plt.axis('off')
plt.tight_layout(pad=0)

# Convert plot to image and write first frame

```

```

fig.canvas.draw()
frame = np.asarray(fig.canvas.buffer_rgba(), dtype=np.uint8)[..., :3]
frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
frame_resized = cv2.resize(frame_bgr, video_size)
vid.write(frame_resized)

# -----insert your occupancy grid mapping parameters here-----

# Log-odds update parameters for the inverse sensor model
log_odds_occ = 0.85
log_odds_free = -0.4
log_odds_min = -5.0
log_odds_max = 5.0

# -----end of your occupancy grid mapping algorithm-----

# Loop over laser scans (every fifth)
for i in range(0, t_laser.shape[0], 5):

    # -----insert your occupancy grid mapping algorithm here-----

    x_r, y_r, theta_r = x_interp[i], y_interp[i], theta_interp[i]

    # loop over beams
    for j in range(npoints):
        r_j = y_laser[i, j]
        if not np.isfinite(r_j):
            continue
        if r_j < r_min_laser or r_j > r_max_laser:
            continue

        # step 1: beam direction in world frame
        angle_j = theta_r + angles[j]
        cos_angle_j = np.cos(angle_j)
        sin_angle_j = np.sin(angle_j)

        # step 2: ray march.
        # 2.1 cells before endpoint -> free
        steps = np.arange(int(r_j / ogres)) * ogres
        x_index = ((x_r + steps * cos_angle_j - ogxmin) / ogres).
            astype(int)
        y_index = ((y_r + steps * sin_angle_j - ogymin) / ogres).
            astype(int)
        valid = (x_index >= 0) & (x_index < ognx) & (y_index >= 0) & (
            y_index < ogny)
        oglo[y_index[valid], x_index[valid]] += log_odds_free
        # 2.2 endpoint cell -> occupied
        x_end = int((x_r + r_j * cos_angle_j - ogxmin) / ogres)
        y_end = int((y_r + r_j * sin_angle_j - ogymin) / ogres)
        if ((0 <= x_end) and (x_end < ognx)) and ((0 <= y_end) and (
            y_end < ogny)):
            oglo[y_end, x_end] += log_odds_occ

        # step 3: clamp + convert
        oglo = np.clip(oglo, log_odds_min, log_odds_max)
        ogp = np.exp(oglo) / (1 + np.exp(oglo))

# -----end of your occupancy grid mapping algorithm-----

```

```

# Draw the map
plt.clf()
plt.imshow(1 - ogp, cmap='gray', origin='lower')
plt.axis('equal')
plt.axis('off')

# Draw the robot
x = (x_interp[i] - ogxmin) / ogres
y = (y_interp[i] - ogymin) / ogres
th = theta_interp[i]
r = 0.15 / ogres

# Draw robot as a circle with heading indicator
circle = patches.Circle((x, y), r, linewidth=2, edgecolor='blue',
                        facecolor=[0.35, 0.35, 0.75])
plt.gca().add_patch(circle)

# Draw heading line
plt.plot([x, x + r * np.cos(th)], [y, y + r * np.sin(th)], 'k-',
        linewidth=2)

# Save the video frame
fig.canvas.draw()
frame = np.asarray(fig.canvas.buffer_rgba(), dtype=np.uint8)[...,
                    :3]
frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
frame_resized = cv2.resize(frame_bgr, video_size)
vid.write(frame_resized)

plt.pause(0.1)

vid.release()
plt.savefig('ass2_q1.png', dpi=100, bbox_inches='tight')
plt.close()

# Save the occupancy grid map
import scipy.io as sio
sio.savemat('occmap.mat', {
    'ogres': ogres,
    'ogxmin': ogxmin,
    'ogxmax': ogxmax,
    'ogymin': ogymin,
    'ogymax': ogymax,
    'ognx': ognx,
    'ogny': ogny,
    'oglo': oglo,
    'ogp': ogp
})

print("Occupancy_grid_mapping_complete!")
print(f"Video_saved_as:_ass2_q1.mp4")
print(f"Map_image_saved_as:_ass2_q1.png")
print(f"Occupancy_grid_data_saved_as:_occmap.mat")

```

Question 2: Particle Filter Localization

```
# =====
# ass3_q2.m
# =====
#
# This assignment will introduce you to the idea of first building an
# occupancy grid then using that grid to estimate a robot's motion
# using a
# particle filter.
#
# There are two questions to complete (5 marks each):
#
#     Question 1: see ass3_q1.m
#     Question 2: code particle filter to localize from known map
#
# Fill in the required sections of this script with your code, run it
# to
# generate the requested plot & movie, then paste the plots into a
# short report
# that includes a few comments about what you've observed. Append
# your
# version of this script to the report. Hand in the report as a PDF
# file
# and the two resulting .mp4 files from Questions 1 and 2.
#
# requires: numpy, scipy, matplotlib, opencv-python, and the data
# files
# 'gazebo.mat', 'occmap.mat'
#
# Steven Waslander March 2026
#

import math

import cv2
import numpy as np
from scipy.io import loadmat

# matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def mat_scalar(value):
    return float(np.asarray(value).squeeze())

def wrap_to_pi(angle):
    return (angle + np.pi) % (2.0 * np.pi) - np.pi

def canvas_to_bgr(fig, frame_size):
    fig.canvas.draw()
    frame = np.asarray(fig.canvas.buffer_rgba(), dtype=np.uint8)[...,
        :3]
    frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
    return cv2.resize(frame_bgr, frame_size)

def draw_robot(ax, x_world, y_world, heading, cell_size, x_min, y_min)
```

```

:
x_cell = (x_world - x_min) / cell_size
y_cell = (y_world - y_min) / cell_size
robot_radius = 0.15 / cell_size
robot = patches.Circle(
    (x_cell, y_cell),
    robot_radius,
    linewidth=2,
    edgecolor="black",
    facecolor=[0.35, 0.35, 0.75],
)
ax.add_patch(robot)
ax.plot(
    [x_cell, x_cell + robot_radius * math.cos(heading)],
    [y_cell, y_cell + robot_radius * math.sin(heading)],
    "k-",
    linewidth=2
)

#
=====

# Question 2: localization from an occupancy grid map using particle
# filter
#
=====

#
# Write a particle filter localization algorithm to localize from the
# laser
# rangefinder readings, wheel odometry, and the occupancy grid map you
# built in Question 1. We will only use two laser scan lines at the
# extreme left and right of the field of view, to demonstrate that the
# algorithm does not need a lot of information to localize fairly well
# . To
# make the problem harder, the below lines add noise to the wheel
# odometry
# and to the laser scans. The plot "ass2_q2_soln.png" shows
# the errors in the estimates produced by wheel odometry alone and by
# the
# particle filter look like as compared to ground truth; we can see
# that
# the errors are much lower when we use the particle filter.

def main():
    # ---- toggle display and recordings ----
    SHOW_PLOT = True          # show live plot window during run
    RECORD_VIDEO = False      # write frames to video file
    # -----

    np.random.seed(1)
    if SHOW_PLOT:
        plt.ion()

    # load the sensor and map data
    gazebo = loadmat("gazebo.mat")
    occmap = loadmat("occmap.mat")

    t_true = gazebo["t_true"].flatten()
    x_true = gazebo["x_true"].flatten()

```



```

y_true = gazebo["y_true"].flatten()
theta_true = gazebo["theta_true"].flatten()
t_odom = gazebo["t_odom"].flatten()
v_odom = gazebo["v_odom"].flatten()
omega_odom = gazebo["omega_odom"].flatten()
t_laser = gazebo["t_laser"].flatten()
y_laser = gazebo["y_laser"].astype(float)
phi_min_laser = mat_scalar(gazebo["phi_min_laser"])
phi_max_laser = mat_scalar(gazebo["phi_max_laser"])
r_max_laser = mat_scalar(gazebo["r_max_laser"])

ogres = mat_scalar(occmap["ogres"])
ogxmin = mat_scalar(occmap["ogxmin"])
ogxmax = mat_scalar(occmap["ogxmax"])
ogymin = mat_scalar(occmap["ogymin"])
ogymax = mat_scalar(occmap["ogymax"])
ogp = occmap["ogp"].astype(float)

# interpolate the noise-free ground-truth at the laser timestamps
numodom = t_odom.shape[0]
t_interp = np.linspace(t_true[0], t_true[numodom - 1], numodom)
x_interp = np.interp(t_laser, t_interp, x_true)
y_interp = np.interp(t_laser, t_interp, y_true)
theta_interp = np.interp(t_laser, t_interp, theta_true)

# interpolate the wheel odometry at the laser timestamps and
# add noise to measurements (yes, on purpose to see effect)
v_interp = np.interp(t_laser, t_interp, v_odom) + 0.2 * np.random.
    randn(t_laser.shape[0])
omega_interp = np.interp(t_laser, t_interp, omega_odom) + 0.04 *
    np.random.randn(t_laser.shape[0])

# add noise to the laser range measurements (yes, on purpose to
    see effect)
# and precompute some quantities useful to the laser
y_laser = y_laser + 0.1 * np.random.randn(*y_laser.shape)
npoints = y_laser.shape[1]
angles = np.linspace(phi_min_laser, phi_max_laser, npoints)
y_laser_max = 5.0

# particle filter tuning parameters (yours may be different)
nparticles = 200
v_noise = 0.2
u_noise = 0.2
omega_noise = 0.04
laser_var = 0.5 ** 2
w_gain = 10.0 * math.sqrt(2.0 * math.pi * laser_var)

# generate an initial cloud of particles
x_particle = x_true[0] + 0.5 * np.random.randn(nparticles)
y_particle = y_true[0] + 0.3 * np.random.randn(nparticles)
theta_particle = theta_true[0] + 0.1 * np.random.randn(nparticles)

# initialize the odom solution
x_odom_only = x_true[0]
y_odom_only = y_true[0]
theta_odom_only = theta_true[0]

pf_err = np.zeros_like(t_laser, dtype=float)
wo_err = np.zeros_like(t_laser, dtype=float)

height, width = ogp.shape

```

```

frame_size = (4 * width, 4 * height)

if RECORD_VIDEO:
    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    video = cv2.VideoWriter("ass2_q2.mp4", fourcc, 10.0,
        frame_size)
else:
    video = None

if SHOW_PLOT or RECORD_VIDEO:
    fig_map, ax_map = plt.subplots(figsize=(10, 8))
    if SHOW_PLOT:
        fig_map.show()
else:
    fig_map, ax_map = None, None

# update the output frae to include the map, robot, measurements,
# filter particles and odom solution
def render_map_frame(scan_index):
    if not SHOW_PLOT and not RECORD_VIDEO:
        return

    # Display the map
    ax_map.clear()
    ax_map.imshow(1.0 - ogp, cmap="gray", origin="lower")
    ax_map.scatter(
        (x_particle - ogxmin) / ogres,
        (y_particle - ogymin) / ogres,
        s=10,
        c=[[0.0, 0.6, 0.0]],
    )

    # Display the odometry-only solution
    ax_map.scatter(
        [(x_odom_only - ogxmin) / ogres],
        [(y_odom_only - ogymin) / ogres],
        s=60,
        c="red",
    )

    # Display the true robot position and laser beams
    x_true_cell = (x_interp[scan_index] - ogxmin) / ogres
    y_true_cell = (y_interp[scan_index] - ogymin) / ogres
    ax_map.scatter(
        [(np.mean(x_particle) - ogxmin) / ogres],
        [(np.mean(y_particle) - ogymin) / ogres],
        s=60,
        c="green",
    )

    # Display the laser beam measurements
    for beam_index in (0, npoints - 1):
        beam_range = y_laser[scan_index, beam_index]
        if np.isfinite(beam_range) and beam_range <= y_laser_max:
            ax_map.plot(
                [x_true_cell, x_true_cell + beam_range / ogres *
                    math.cos(theta_interp[scan_index] + angles[
                        beam_index])],
                [y_true_cell, y_true_cell + beam_range / ogres *
                    math.sin(theta_interp[scan_index] + angles[
                        beam_index])],
                "m-",
                linewidth=1,

```

```

    )

    # Display the robot
    draw_robot(ax_map, x_interp[scan_index], y_interp[scan_index],
               theta_interp[scan_index], ogres, ogxmin, ogymmin)
    ax_map.set_aspect("equal")
    ax_map.axis("off")
    fig_map.tight_layout(pad=0)

    if SHOW_PLOT:
        fig_map.canvas.draw_idle()
        fig_map.canvas.flush_events()
        plt.pause(0.001)

    if RECORD_VIDEO:
        video.write(canvas_to_bgr(fig_map, frame_size))

render_map_frame(0)

beam_indices = (0, npoints - 1)
normalizer = math.sqrt(2.0 * math.pi * laser_var)

# Main filter loop
for i in range(1, t_laser.shape[0]):
    dt = t_laser[i] - t_laser[i - 1]

    v = v_interp[i]
    omega = omega_interp[i]
    x_odom_only = x_odom_only + dt * v * math.cos(theta_odom_only)
    y_odom_only = y_odom_only + dt * v * math.sin(theta_odom_only)
    theta_odom_only = wrap_to_pi(theta_odom_only + dt * omega)

    w_particle = np.ones(nparticles, dtype=float)

    # Loop over particles
    for n in range(nparticles):

        # propagate the particle forward in time using wheel
        # odometry
        v_sample = v_interp[i] + v_noise * np.random.randn()
        u_sample = u_noise * np.random.randn()
        omega_sample = omega_interp[i] + omega_noise * np.random.
            randn()

        x_particle[n] = x_particle[n] + dt * (
            v_sample * math.cos(theta_particle[n]) - u_sample *
            math.sin(theta_particle[n])
        )
        y_particle[n] = y_particle[n] + dt * (
            v_sample * math.sin(theta_particle[n]) + u_sample *
            math.cos(theta_particle[n])
        )
        theta_particle[n] = wrap_to_pi(theta_particle[n] + dt *
            omega_sample)

    for beam_index in beam_indices:
        measured_range = y_laser[i, beam_index]
        if not np.isfinite(measured_range) or measured_range >
            y_laser_max or measured_range <= 0.0:
            continue

    # -----insert your particle filter weight calculation

```

```

        here -----

# step 1: ray cast. expected range from particle
position
beam_angle = theta_particle[n] + angles[beam_index]
cos_beam_angle = math.cos(beam_angle)
sin_beam_angle = math.sin(beam_angle)

expected_range = y_laser_max # default: no hit
d = 0.0
while d < y_laser_max:
    # current cell along ray
    x_index = int((x_particle[n] + d * cos_beam_angle
        - ogxmin) / ogres)
    y_index = int((y_particle[n] + d * sin_beam_angle
        - ogymin) / ogres)

    if x_index < 0 or x_index >= width or y_index < 0
        or y_index >= height:
        break

    # if hit occupied cell, update expected range and
    break
    if ogp[y_index, x_index] > 0.5:
        expected_range = d
        break

    d += ogres

# step 2: residual
residual = measured_range - expected_range

# step 3: update weight
w_particle[n] *= w_gain * math.exp(-residual**2 / (2.0
    * laser_var))

# -----end of your particle filter weight calculation
-----

weight_sum = np.sum(w_particle)
if not np.isfinite(weight_sum) or weight_sum <= 0.0:
    w_particle.fill(1.0 / nparticles)
else:
    w_particle /= weight_sum

cumulative_weights = np.cumsum(w_particle)
start = np.random.rand() / nparticles
targets = start + np.arange(nparticles) / nparticles
indices = np.searchsorted(cumulative_weights, targets, side="
    left")
indices = np.clip(indices, 0, nparticles - 1)

x_particle = x_particle[indices]
y_particle = y_particle[indices]
theta_particle = theta_particle[indices]

pf_err[i] = math.sqrt((np.mean(x_particle) - x_interp[i]) ** 2
    + (np.mean(y_particle) - y_interp[i]) ** 2)
wo_err[i] = math.sqrt((x_odom_only - x_interp[i]) ** 2 + (
    y_odom_only - y_interp[i]) ** 2)

```

```

        render_map_frame(i)

    if RECORD_VIDEO:
        video.release()
    if SHOW_PLOT or RECORD_VIDEO:
        plt.close(fig_map)

    fig_err, ax_err = plt.subplots(figsize=(8, 4.5))
    ax_err.plot(t_laser, pf_err, "g-", label="particle_filter")
    ax_err.plot(t_laser, wo_err, "r-", label="odom")
    ax_err.set_xlabel("t[s]")
    ax_err.set_ylabel("error[m]")
    ax_err.set_title("error(estimate-true)")
    ax_err.legend(loc="upper_left")
    fig_err.tight_layout()
    fig_err.savefig("ass2_q2.png", dpi=150)
    plt.close(fig_err)

    print("Particle_filter_localization_complete!")
    if RECORD_VIDEO:
        print("Video_saved_as:_ass2_q2.mp4")
    print("Error_plot_saved_as:_ass2_q2.png")

if __name__ == "__main__":
    main()

```